



ADR-008: Hash-chained, append-only audit log

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

- **Status:** Accepted
- **Date:** 2026-10-07
- **Related:** [ADR-003](#), [Security architecture §12](#)

Context and problem statement

Insurance regulators, TASCO internal audit and the DPO need to know who did what, when, and to which entity, with assurance that the record was not altered afterwards. This covers:

- rule approvals (tariffs, scripts, commission);
- logins and MFA failures;
- profile views with or without PII;
- consent changes and DSAR actions;
- orders and claim transitions;
- partner key issuance.

Ordinary application logs can be edited or deleted by anyone with access to the database or the log store.

Decision drivers

- Tampering must be detectable, not just prevented.
- The log must be append-only at the database level.
- Verification must be cheap and available to auditors in the product.
- No PII in audit details (ids only).

Considered options

1. **Hash chain in Postgres:** each entry stores `prev_hash` and `hash = SHA-256(canonical([prevHash, id, at, actor, action, entityType, entityId, details]))`; a trigger forbids UPDATE and DELETE; appends are serialised with an advisory lock.
2. Ship logs to SIEM / WORM storage only.
3. Ledger database (Amazon QLDB, Azure Confidential Ledger) or blockchain.

Decision outcome

Chosen option: **1**, with the SIEM/WORM copy added as defence in depth (target).

- `auditChain.js`:
 - `canonical()` serialises with recursively sorted keys, so hashes survive jsonb key reordering;
 - `entryHash()` computes the entry hash;
 - `verifyChain()` reports `{ok, brokenAt, reason}` or `{ok, entries, head}`.
- `postgresStore.audit.append` runs `pg_advisory_xact_lock(724001)` in a transaction, reads the last hash by `seq`, inserts the new entry and commits. The chain stays linear across replicas.
- `db/migrations/001_init.sql`: the `audit_log` table has `seq` `bigserial`, unique `id` and unique `hash`. Trigger `trg_audit_log_immutable` raises on UPDATE OR DELETE.
- API: GET `/api/audit` (search), GET `/api/audit/verify`, and `/api/dashboard/governance` includes the chain status. Permission: `audit:read`.
- `auditService` writes ids only. Details carry ids, counts, reasons and checksums, never names or phones. Login records include the client IP.

Consequences

- Good: editing or deleting any row breaks verification at that index. Inserting a forged row requires recomputing every later hash, which is detectable against an external anchor (below).
- Good: both stores share the same code, so the property is tested in unit tests.
- Bad / gaps:
 - **TRUNCATE bypasses row-level triggers.** `postgresStore.reset()` truncates `audit_log` (used by tests). In production the application DB role must **not** own the table and must not hold `TRUNCATE` or `ALTER / DISABLE TRIGGER`. Use a separate migration/owner role, and grant the app role `INSERT`, `SELECT` only on `audit_log` (and `USAGE` on its sequence). Add a statement-level `BEFORE TRUNCATE` trigger.
 - **Whole-chain truncation or rewrite by a DB superuser is undetectable from inside the DB.** **Target:** anchor the head hash every hour to an external WORM store (object lock / SIEM). `verify` then checks against the last anchor.
 - **Cost of verification.** `verify()` loads the whole table and runs on every governance dashboard request. At millions of rows this must become **incremental**: store verified checkpoints (`seq`, `hash`) and verify from the last checkpoint only.
 - **Throughput.** A global advisory lock serialises appends. That is fine for current audit points; if needed, move to per-partition chains or periodic Merkle roots.
 - **Audit writes are not in the same transaction as the business change** ([ADR-004](#) gap). A crash can leave a change without its audit entry.
 - **Retention:** 10 years, archive and never delete early (`config/rules/retention.json`). The archival pipeline is planned.

Pros and cons of the options

SIEM/WORM only. Strong retention. However, it is not queryable in-product, and integrity depends on a separate system.

Ledger DB / blockchain. Strong cryptographic guarantees. However, it adds a new vendor or runtime and data-residency questions in Vietnam (**confirm with TASCO legal**), and it is unnecessary at this scale.